Set A

1) Display date, time, logged-in user, and shell name

```
#!/bin/bash

echo "Current Date & Time: $(date)"
echo "Logged-in User: $(whoami)"
echo "Shell being used: $SHELL"
```

---

2) Multiplication table (1–10 for numbers 1–5), save to file and display

```
#!/bin/bash

output_file="multiplication_table.txt"

> "$output_file"

for i in {1..5}
do
  for j in {1..10}
  do
    echo "$i x $j = $((i*j))" >> "$output_file"
  done
  echo "-------------------" >> "$output_file"
done

cat "$output_file"
```

---

3) Recursively find `.txt` files and count lines containing "error"

```
#!/bin/bash

dir="$1"
```

```
report="report.txt"

> "$report"

find "$dir" -type f -name "*.txt" | while read file
do
  count=$(grep -i "error" "$file" | wc -l)
  echo "$file : $count" >> "$report"
done

echo "Report saved to $report"
```

Set B

4) Check host/IP using ping and log result with timestamp

```
#!/bin/bash

host="$1"
logfile="/home/$USER/network_log.txt"

timestamp=$(date "+%Y-%m-%d %H:%M:%S")

if ping -c 1 "$host" &> /dev/null
then
  echo "$timestamp - SUCCESS: $host is reachable" >> "$logfile"
else
  echo "$timestamp - FAILURE: $host is unreachable" >> "$logfile"
fi
```

---

5) Monitor CPU usage with conditions

```
#!/bin/bash

iterations=$1
i=1

while [ $i -le $iterations ]
do
  cpu=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}')
```

```
  echo "Iteration $i: CPU Usage = $cpu%"

  if (( $(echo "$cpu < 10" | bc -l) ))
  then
    ((i++))
    continue
  fi

  if (( $(echo "$cpu > 90" | bc -l) ))
  then
    echo "CPU exceeded 90%. Exiting."
    break
  fi

  echo "Logged CPU usage: $cpu%"
  ((i++))
done
```

---

6) Extract IPs, timestamps, and errors using regex

```
#!/bin/bash

logfile="$1"

echo "Valid IP Addresses:"
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$logfile"

echo "Timestamps (YYYY-MM-DD HH:MM:SS):"
grep -Eo '[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}' "$logfile"

echo "Error Messages:"
grep -i "error" "$logfile"
```

Set C

7) Check if a number is prime using a function

```
#!/bin/bash

is_prime() {
  num=$1
```

```
  if [ "$num" -le 1 ]; then
    return 1
  fi

  for ((i=2; i<=num/2; i++))
  do
    if (( num % i == 0 ))
    then
      return 1
    fi
  done
  return 0
}

read -p "Enter a number: " n

if is_prime "$n"
then
  echo "$n is a prime number"
else
  echo "$n is not a prime number"
fi
```

---

8) Monitor directory size and log warning

```
#!/bin/bash

dir="$1"
limit_mb=500
logfile="/home/$USER/dir_size_log.txt"

size=$(du -sm "$dir" | cut -f1)
timestamp=$(date "+%Y-%m-%d %H:%M:%S")

if [ "$size" -gt "$limit_mb" ]
then
  echo "$timestamp - WARNING: $dir size = ${size}MB exceeds ${limit_mb}MB" >> "$logfile"
fi
```

---

9) AWK script for CSV processing

```
#!/bin/bash
```

```
file="employees.csv"

echo "Average Salary per Department:"
awk -F, '
NR>1 {
  sum[$2]+=$3;
  count[$2]++;
}
END {
  for (dept in sum)
    print dept, sum[dept]/count[dept];
}' "$file"

echo "Employees with >5 years:"
awk -F, '
NR>1 {
  split($4, d, "-");
  year=d[1];
  if ((strftime("%Y") - year) > 5)
    print $1;
}' "$file"
```

---

10) Incremental backup + cron job

Backup Script

```
#!/bin/bash

src_dir="/home/$USER/source"
backup_dir="/home/$USER/backups"
logfile="/home/$USER/userlog.txt"

timestamp=$(date "+%Y-%m-%d_%H-%M-%S")

mkdir -p "$backup_dir"

tar -czf "$backup_dir/backup_$timestamp.tar.gz" \
$(find "$src_dir" -type f -mtime -1) 2>> "$logfile"

if [ $? -ne 0 ]
then
  echo "$(date) - Backup FAILED" >> "$logfile"
else
  echo "$(date) - Backup SUCCESS" >> "$logfile"
```

```
fi
```

Cron Job (run daily at 1:00 AM)

```
crontab -e
```

Add this line:

```
0 1 * * * /bin/bash /home/$USER/backup_script.sh
```

Below are **Practical 2 answers** with each **question + exact script/commands** (clean and exam-ready).

Set A

1) Backup directory with timestamped folder

```
#!/bin/bash

SOURCE="/home/user/data"
DESTINATION="/home/user/backup"
DATE=$(date "+%Y-%m-%d_%H-%M-%S")

mkdir -p "$DESTINATION/$DATE"

cp -r "$SOURCE"/* "$DESTINATION/$DATE"

echo "Backup completed on $DATE"
```

---

2) Check disk usage and warn if >80%

```
#!/bin/bash

THRESHOLD=80

df -h | grep -vE '^Filesystem|tmpfs|cdrom' | while read line
do
  usage=$(echo $line | awk '{print $5}' | tr -d '%')
```

```
  partition=$(echo $line | awk '{print $1}')

  if [ "$usage" -gt "$THRESHOLD" ]
  then
    echo "Warning: Disk usage on $partition is ${usage}%"
  fi
done
```

---

3) Check Jenkins service and start if not running

```
#!/bin/bash

SERVICE="jenkins"

if systemctl is-active --quiet $SERVICE
then
  echo "$SERVICE is running"
else
  echo "$SERVICE is not running. Starting service..."
  systemctl start $SERVICE
  echo "$SERVICE started"
fi
```

Set B

4) Check reachability of google.com and log result

```
#!/bin/bash

HOST="google.com"
OUTPUT_FILE="ping_results.log"

if ping -c 1 $HOST &> /dev/null
then
  echo "$HOST is reachable" >> $OUTPUT_FILE
else
  echo "$HOST is not reachable" >> $OUTPUT_FILE
fi
```

---

5) Backup MySQL database with timestamp

```
#!/bin/bash

DB_NAME="school"
BACKUP_DIR="/backups"
DATE=$(date "+%Y-%m-%d_%H-%M-%S")

read -s -p "Enter MySQL password: " DB_PASS
echo

mkdir -p "$BACKUP_DIR"

mysqldump -u root -p$DB_PASS $DB_NAME > "$BACKUP_DIR/${DB_NAME}_$DATE.sql"

echo "Database backup completed: $BACKUP_DIR/${DB_NAME}_$DATE.sql"
```

---

6) Store system uptime with timestamp

```
#!/bin/bash

LOG_FILE="system_uptime.log"

echo "$(date "+%Y-%m-%d %H:%M:%S") - $(uptime -p)" >> $LOG_FILE
```

---

7) Display all TCP and UDP listening ports

```
netstat -tuln | grep LISTEN
```

(Alternative modern command)

```
ss -tuln
```

Set C

8) Update, upgrade, clean packages with messages

```
#!/bin/bash
```

```
echo "Updating package list..."
sudo apt-get update

echo "Upgrading packages..."
sudo apt-get upgrade -y

echo "Removing unnecessary packages..."
sudo apt-get autoremove -y

echo "Cleaning cache..."
sudo apt-get clean

echo "System packages updated and cleaned up"
```

---

9) Check response time of URLs

```
#!/bin/bash

URLS=("https://example.com" "https://google.com")

for URL in "${URLS[@]}"
do
  RESPONSE_TIME=$(curl -o /dev/null -s -w '%{time_total}\n' $URL)
  echo "Response time for $URL: $RESPONSE_TIME seconds"
done
```

---

10) Top 10 memory-consuming processes

```
ps aux --sort=-%mem | head -n 10
```

Here are **Practical 3 (Uptime + Networking + SCP) answers** with **questions + exact scripts/commands**.

Set A

1) Monitor uptime continuously and store in TSV

```
#!/bin/bash
```

```
FILE="uptime_data.tsv"

touch "$FILE"

while true
do
  TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
  UPTIME=$(uptime -p)

  echo -e "$TIMESTAMP\t$UPTIME" >> "$FILE"

  # Sort in descending order (latest first)
  sort -r "$FILE" -o "$FILE"

  sleep 300   # 5 minutes
done
```

---

2) TCP Client-Server (Python)

Server (EC2)

```
import socket

HOST = '0.0.0.0'
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print("Server listening...")

while True:
    conn, addr = server.accept()
    print(f"Connected by {addr}")

    with open("uptime_data.tsv", "r") as f:
        latest = f.readline()

    conn.sendall(latest.encode())
    conn.close()
```

Client (Local Machine)

```
import socket

HOST = 'EC2_PUBLIC_IP'   # replace
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

data = client.recv(1024)
print("Latest Uptime:", data.decode())

client.close()
```

---

3) UDP Client-Server (Python)

Server

```
import socket

HOST = '0.0.0.0'
PORT = 5001

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))

print("UDP Server listening...")

while True:
    data, addr = server.recvfrom(1024)

    with open("uptime_data.tsv", "r") as f:
        latest = f.readline()

    server.sendto(latest.encode(), addr)
```

Client

```
import socket

HOST = 'EC2_PUBLIC_IP'   # replace
PORT = 5001

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
client.sendto(b"Request", (HOST, PORT))
data, _ = client.recvfrom(1024)

print("Latest Uptime:", data.decode())
```

Set B

4) FTP Client-Server (Python)

Server (simple FTP using pyftpdlib)

```
pip install pyftpdlib

from pyftpdlib.authorizers import DummyAuthorizer
from pyftpdlib.handlers import FTPHandler
from pyftpdlib.servers import FTPServer

authorizer = DummyAuthorizer()
authorizer.add_user("user", "12345", ".", perm="elradfmw")

handler = FTPHandler
handler.authorizer = authorizer

server = FTPServer(("0.0.0.0", 2121), handler)
print("FTP Server running...")
server.serve_forever()
```

Client

```
from ftplib import FTP

ftp = FTP()
ftp.connect("127.0.0.1", 2121)
ftp.login("user", "12345")

with open("uptime_data.tsv", "wb") as f:
    ftp.retrbinary("RETR uptime_data.tsv", f.write)

print("File downloaded")

ftp.quit()
```

Set C

5) SCP Commands

Replace:

- `EC2_IP` → your instance IP
- `key.pem` → your AWS key

I) Copy single file (EC2 → Local)

scp -i key.pem ubuntu@EC2_IP:/home/ubuntu/example.txt /local_backup/

II) Copy multiple files

scp -i key.pem ubuntu@EC2_IP:/home/ubuntu/files/{file1.txt,file2.txt,file3.txt} /local_files/

III) Copy entire directory

scp -i key.pem -r ubuntu@EC2_IP:/var/www /local/web_backup/

IV) Local → EC2 (single file)

scp -i key.pem /local_reports/report.csv ubuntu@EC2_IP:/home/username/reports/

V) Copy multiple PDFs (Local → EC2)

scp -i key.pem /local_documents/{doc1.pdf,doc2.pdf,doc3.pdf} ubuntu@EC2_IP:/home/username/docs/

VI) Copy entire directory (Local → EC2)

scp -i key.pem -r /images ubuntu@EC2_IP:/home/username/media/

VII) SCP with compression

scp -i key.pem -C largefile.zip ubuntu@EC2_IP:/home/username/

VIII) Preserve permissions

scp -i key.pem -rp /local_dir ubuntu@EC2_IP:/home/username/

IX) Verbose mode

scp -i key.pem -v file.txt ubuntu@EC2_IP:/home/username/

Here are **Practical 4 (Terraform + EC2 Deployment) answers** with **questions + exact configs + commands**.

Set A

1) Manual setup of Apache project on Ubuntu EC2

Commands (run on EC2 instance via SSH)

```
# Update system
sudo apt update

# Install Apache2
sudo apt install apache2 -y

# Start and enable Apache
sudo systemctl start apache2
sudo systemctl enable apache2

# Download sample project (example)
cd /tmp
wget https://www.tooplate.com/zip-templates/2135_mini_finance.zip

# Install unzip
sudo apt install unzip -y
```

```
# Extract files
unzip 2135_mini_finance.zip

# Copy project to web root
sudo cp -r 2135_mini_finance/* /var/www/html/

# Set permissions
sudo chown -R www-data:www-data /var/www/html/

# Restart Apache
sudo systemctl restart apache2

echo "Project deployed successfully"
```

👉 Access using: `http://<EC2_PRIVATE_IP>`

---

2) Terraform configuration to create EC2

main.tf

```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "example" {
  ami           = "ami-00bb6a80f"
  instance_type = "t2.micro"

  tags = {
    Name = "ExampleAppServerInstance"
  }
}
```

Terraform Commands

```
terraform init
terraform validate
terraform plan
terraform apply
```

Set B

3) Use variables.tf for Name tag

variables.tf

```
variable "instance_name" {
  description = "EC2 instance name"
  type        = string
  default     = "MyEC2Instance"
}
```

Updated main.tf

```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "example" {
  ami           = "ami-00bb6a80f"
  instance_type = "t2.micro"

  tags = {
    Name = var.instance_name
  }
}
```

---

4) outputs.tf (Display instance details)

```
output "instance_id" {
  value = aws_instance.example.id
}

output "public_ip" {
  value = aws_instance.example.public_ip
}
```

Set C

5) Full Terraform deployment (EC2 + Security Group + User Data)

---

main.tf (Complete Configuration)

```
provider "aws" {
  region = "ap-south-1"
}

# Security Group
resource "aws_security_group" "web_sg" {
  name = "web_sg"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# EC2 Instance
resource "aws_instance" "web" {
  ami           = "ami-00bb6a80f"
  instance_type = "t2.micro"
  key_name      = "mykey"

  security_groups = [aws_security_group.web_sg.name]

  root_block_device {
    volume_size = 10
  }
```

```
  user_data = <<-EOF
            #!/bin/bash
            apt update
            apt install apache2 wget unzip vim -y
            cd /tmp
            wget https://www.tooplate.com/zip-templates/2135_mini_finance.zip
            unzip 2135_mini_finance.zip
            cp -r 2135_mini_finance/* /var/www/html/
            systemctl start apache2
            systemctl enable apache2
            EOF

  tags = {
    Name = "WebAppInstance"
  }
}
```

---

i) Purpose of Security Group

- Acts as a **virtual firewall**
- Allows:
  - **SSH (22)** → Remote access
  - **HTTP (80)** → Web access
- Blocks all other unwanted traffic
- Controls inbound/outbound network rules

---

ii) Terraform Execution Steps

terraform init
terraform validate
terraform plan
terraform apply

---

iii) After Apply

Verify

- Go to **AWS Console** → **EC2** → **Instances**
- Check instance is **running**

Access Web App

http://<PUBLIC_IP>

Here is Practical 5 (Git & GitHub Workflows) with questions + exact commands (exam-ready).

Set A

1A) Clone a public repository

git clone https://github.com/username/repository.git

cd repository

1B) Create repo + modify README + commit

 # Initialize repo (if local)

git init

 # Create README

echo "# My Project" > README.md

echo "This project demonstrates Git workflow" >> README.md

 # Add & commit

git add README.md

git commit -m "Added README with project description"

1C) Configure username and email globally

```
git config --global user.name "Your Name"

git config --global user.email "your@email.com"
```

1D) Pull README from remote repository

```
git pull origin main
```

1E) Edit README + create index.html + stage

```
# Edit README

echo "Updated README content" >> README.md


# Create index.html

echo "<html><body><h1>Welcome</h1></body></html>" > index.html


# Stage files

git add README.md index.html
```

1F) Commit and push using upstream

```
git commit -m "Updated README and added index.html"


git push -u origin main
```

Set B

2) Branching, Commits, Push (C0, C1, C2)

```
# Initial commits

echo "<h1>Initial</h1>" > README.md

git add README.md

git commit -m "C0: Added h1 tag in README"


# Create new file

echo "Text file content" > file.txt

git add file.txt

git commit -m "C1: Created new txt file"


# Modify file

echo "Updated content" >> file.txt

git add file.txt

git commit -m "C2: Modified txt file"


# Push to remote main

git push origin main
```

Set C

b) Create branch iss53, commit C3, diff, push

```
# Create branch

git checkout -b iss53


# Create index.html and modify README

echo "<html>ISS53</html>" > index.html

echo "Update for iss53" >> README.md


git add .

git commit -m "C3: Added index.html and updated README in iss53"


# View differences

git diff main iss53


# Push branch

git push origin iss53
```

---

c) Create hotfix branch and commit C4

```
# Switch to main

git checkout main


# Create hotfix branch

git checkout -b hotfix
```

```
# Update file
echo "Hotfix update" >> file.txt

git add file.txt
git commit -m "C4: Updated txt file in hotfix"

# Push hotfix branch
git push origin hotfix
```

---

d) Merge hotfix into main and delete branch

```
git checkout main

git merge hotfix

# Delete branch locally
git branch -d hotfix

# Delete remote branch
git push origin --delete hotfix
```

---

e) Merge iss53 into main and delete branch

```
git checkout main
```

git merge iss53

 # Delete branch locally

git branch -d iss53

 # Delete remote branch

git push origin --delete iss53

Here is **Practical 6 (Docker Commands & Containerization)** with **questions + commands + short descriptions (exam-ready)**.

Set A

1A) Pull Ubuntu image (20.04)

Command:

docker pull ubuntu:20.04

Description: Downloads Ubuntu 20.04 image from Docker Hub to local system.

1B) List all images

Command:

docker images

Description: Displays all locally available Docker images.

1C) Pull Ubuntu latest image

Command:

docker pull ubuntu:latest

Description: Downloads the latest Ubuntu image.

1D) Remove Ubuntu 20.04 image

Command:

docker rmi ubuntu:20.04

Description: Deletes the specified image from local storage.

1E) Run container (detached + interactive)

Command:

docker run -dit --name ubuntu-os ubuntu:latest

Description: Creates and runs a container in detached mode with interactive terminal.

1F) Stop container

Command:

docker stop ubuntu-os

Description: Stops the running container.

1G) Restart container

Command:

docker start ubuntu-os

Description: Restarts the stopped container.

1H) List all containers

Command:

docker ps -a

Description: Shows all containers (running + stopped).

1I) Create directory inside running container

Command:

docker exec ubuntu-os mkdir /myfolder

Description: Creates a directory inside the container.

1J) Verify directory creation

Command:

docker exec ubuntu-os ls /

Description: Lists root directory to confirm folder creation.

1K) Stop container

Command:

docker stop ubuntu-os

Description: Stops the container.

1L) Remove all containers

Command:

```
docker rm $(docker ps -aq)
```

Description: Deletes all containers.

Set B

2A) Pull PostgreSQL latest image

Command:

```
docker pull postgres:latest
```

Description: Downloads latest PostgreSQL image.

2B) Run PostgreSQL container

Command:

```
docker run -d --name postgres-user1 \
-e POSTGRES_PASSWORD=pass123 \
-p 5432:5432 postgres
```

Description: Runs PostgreSQL container with environment variables and port mapping.

2C) Connect to PostgreSQL

Command:

docker exec -it postgres-user1 psql -U postgres

Description: Opens PostgreSQL CLI inside container.

2D) Create database

Command:

CREATE DATABASE mydb;

Description: Creates a new database in PostgreSQL.

2E) Stop container

Command:

docker stop postgres-user1

Description: Stops PostgreSQL container.

2F) Remove container

Command:

docker rm postgres-user1

Description: Deletes PostgreSQL container.

Set C

3) Dockerfile for Node.js HTTP Server

Dockerfile

```
FROM node:18

WORKDIR /app

COPY . .

RUN npm install

EXPOSE 3000

CMD ["node", "app.js"]
```

Build Image

```
docker build -t node-app .
```

Run Container

```
docker run -d -p 3000:3000 node-app
```

---

4) Push Image to Docker Hub

A) Tag image

```
docker tag node-app username/node-app
```

B) Login to Docker Hub

```
docker login
```

C) Push image

```
docker push username/node-app
```

Here is **Practical 7 (Mongo + Mongo-Express using Docker & Docker Compose)** with **questions + exact commands/config (exam-ready)**.

Set A

1A) Run Mongo container with env, port, name, custom network

Step 1: Create custom network

docker network create mongo-net

Step 2: Run Mongo container

```
docker run -d \
--name mongo-db \
--network mongo-net \
-e MONGO_INITDB_ROOT_USERNAME=admin \
-e MONGO_INITDB_ROOT_PASSWORD=admin123 \
-p 27017:27017 \
mongo
```

Description:

- `--name mongo-db` → container name
- `--network mongo-net` → custom network
- `-e` → environment variables (credentials)
- `-p` → port mapping
- `mongo` → image

Set B

1B) Run mongo-express container + connect to Mongo

```
docker run -d \
--name mongo-express \
--network mongo-net \
-e ME_CONFIG_MONGODB_ADMINUSERNAME=admin \
-e ME_CONFIG_MONGODB_ADMINPASSWORD=admin123 \
-e ME_CONFIG_MONGODB_SERVER=mongo-db \
-p 8081:8081 \
```

mongo-express

How containers are interconnected

- Both containers use **same network (`mongo-net`)**
- `ME_CONFIG_MONGODB_SERVER=mongo-db` → uses container name as hostname
- Docker DNS resolves `mongo-db`

Access in browser

http://localhost:8081

Set C

2A & 2B) Using docker-compose.yml

docker-compose.yml

```
version: '3.8'

services:
  mongo:
    image: mongo
    container_name: mongo-db
    restart: always
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: admin123
    ports:
      - "27017:27017"

  mongo-express:
    image: mongo-express
    container_name: mongo-express
    restart: always
    depends_on:
      - mongo
    environment:
      ME_CONFIG_MONGODB_ADMINUSERNAME: admin
      ME_CONFIG_MONGODB_ADMINPASSWORD: admin123
```

```
    ME_CONFIG_MONGODB_SERVER: mongo
  ports:
    - "8081:8081"
```

Run docker-compose

docker-compose up -d

Stop containers

docker-compose down

How it works

- Docker Compose creates a **default network automatically**
- Services communicate using service names (`mongo`)
- `depends_on` ensures Mongo starts before mongo-express
- Browser access:

http://localhost:8081

Below is a **complete list of important commands used across Practicals (Git, Docker, Terraform, Linux, Networking, Mongo, etc.) with their exact use** — organized for quick revision and viva.

🟢 1. Basic Linux Commands

Command	Use
`date`	Show current date and time
`whoami`	Show logged-in user
`uptime`	Show system uptime
`uptime -p`	Human-readable uptime

`df -h`	Disk usage in human-readable format
`du -sm`	Directory size in MB
`grep`	Search text patterns
`awk`	Process and extract text
`find`	Search files in directories
`sort`	Sort file content
`wc -l`	Count lines
`mkdir -p`	Create directory recursively
`cp -r`	Copy directories
`chmod`	Change permissions
`chown`	Change ownership

🟢 2. Git Commands

Command	Use
`git init`	Initialize repository
`git clone <url>`	Clone remote repo
`git config --global user.name/email`	Set identity
`git status`	Check repo status
`git add <file>`	Stage changes
`git commit -m "msg"`	Commit changes
`git push origin main`	Push to remote
`git push -u origin main`	Set upstream
`git pull origin main`	Fetch + merge

`git branch`	List branches
`git checkout -b <branch>`	Create branch
`git checkout <branch>`	Switch branch
`git merge <branch>`	Merge branch
`git diff`	Show differences
`git log --oneline`	Commit history
`git branch -d <branch>`	Delete branch
`git push origin --delete <branch>`	Delete remote branch

🟢 3. Docker Commands

🔹 Images

Command	Use
`docker pull <image>`	Download image
`docker images`	List images
`docker rmi <image>`	Remove image

🔹 Containers

Command	Use
`docker run -d`	Run container (detached)
`docker run -it`	Interactive container
`docker run -dit --name`	Named container

`docker ps`	Running containers
`docker ps -a`	All containers
`docker stop <name>`	Stop container
`docker start <name>`	Start container
`docker restart <name>`	Restart container
`docker rm <name>`	Remove container
`docker rm $(docker ps -aq)`	Remove all containers

🔹 Container Interaction

Command	Use
`docker exec -it <name> bash`	Access container shell
`docker exec <name> mkdir`	Run command inside container
`docker logs <id>`	View logs

🔹 Networking

Command	Use
`docker network create <name>`	Create network
`--network <name>`	Attach container to network

🔹 Build & Push

Command	Use
`docker build -t <name>`	Build image

`docker tag`	Tag image
`docker login`	Login to Docker Hub
`docker push`	Push image

🔹 Docker Compose

Command	Use
`docker-compose up -d`	Start services
`docker-compose down`	Stop services

🟢 4. PostgreSQL (Docker)

Command	Use
`docker exec -it <container> psql -U postgres`	Open DB shell
`CREATE DATABASE dbname;`	Create database

🟢 5. Mongo + Mongo Express

Command	Use
`docker run mongo`	Run Mongo container
`docker run mongo-express`	Run UI
`ME_CONFIG_MONGODB_SERVER`	Connect to Mongo
`docker network create`	Enable container communication

🟢 6. Terraform Commands

Command	Use
`terraform init`	Initialize project
`terraform validate`	Validate config
`terraform plan`	Preview changes
`terraform apply`	Create resources
`terraform destroy`	Delete resources

🟢 7. Networking Commands

Command	Use
`ping <host>`	Check reachability
`netstat -tuln`	Show open ports
`ss -tuln`	Modern netstat
`curl`	Fetch URL data

🟢 8. SCP Commands

Command	Use
`scp file user@host:path`	Copy file

`scp -r`	Copy directory
`scp -i key.pem`	Use SSH key
`scp -C`	Enable compression
`scp -v`	Verbose mode
`scp -p`	Preserve permissions

🟢 9. AWS / EC2 Related

Command	Use
`ssh -i key.pem ubuntu@ip`	Connect to EC2
`apt update`	Update packages
`apt install apache2`	Install Apache
`systemctl start apache2`	Start service
`systemctl enable apache2`	Auto-start

🟢 10. File Processing (Shell Scripts)

Command	Use
`grep -i`	Case-insensitive search
`awk '{print $1}'`	Extract column
`tr -d '%'`	Remove characters
`while read`	Loop through lines

`bc`	Perform calculations

🟢 11. Backup & Cron

Command	Use
`tar -czf`	Create archive
`find -mtime -1`	Files modified in 24h
`crontab -e`	Edit cron jobs
`0 1 * * *`	Run daily at 1 AM

🟢 12. Process Monitoring

Command	Use
`top`	Live CPU usage
`ps aux`	List processes
`--sort=-%mem`	Sort by memory
`head -n 10`	Top 10 results